

Compiler Design Lab Practice (Part 2)

Advanced 30 Problems & Solutions (31 to 60)

31. Write a lex program to recognize Bangladeshi Mobile Numbers (+8801...).

```
%{
#include
%}
%%
(\\+88)?01[3-9][0-9]{8} { printf("Valid BD Mobile Number: %s\\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

32. Write a lex program to count words, lines, and characters in a file.

```
%{
int wc=0, lc=0, cc=0;
%}
%%
\\n { lc++; cc++; }
[ \\t] { cc++; }
[^ \\t\\n]+ { wc++; cc += yyleng; }
%%
int main() {
    yylex();
    printf("Lines: %d, Words: %d, Characters: %d\\n", lc, wc, cc);
    return 0;
}
int yywrap() { return 1; }
```

33. Write a lex program to identify and count Single-line and Multi-line Comments.

```
%{
int sl=0, ml=0;
%}
%%
"//" .* { sl++; printf("Single-line comment\\n"); }
"/*"([\\*]|\\*+[^*/])*\\*+ "/" { ml++; printf("Multi-line comment\\n"); }
.|
\\n { /* ignore other text */ }
%%
int main() {
    yylex();
    printf("Single-line: %d, Multi-line: %d\\n", sl, ml);
    return 0;
}
int yywrap() { return 1; }
```

34. Write a lex program to recognize 'to be' verbs (am, is, are, was, were).

```
%{
#include
%}
%%
\b(am|is|are|was|were|AM|IS|ARE|WAS|WERE)\b { printf("To-Be Verb: %s\n",
yytext); }
.|
\n { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

35. Write a lex program to differentiate Integer, Floating point, and Exponential numbers.

```
%{
#include
%}
%%
[0-9]+ { printf("Integer: %s\n", yytext); }
[0-9]+\.[0-9]+ { printf("Floating Point: %s\n", yytext); }
[0-9]+(\.[0-9]+)?[eE][+-]?[0-9]+ { printf("Exponential: %s\n", yytext); }
.|
\n { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

36. Write a lex program to check valid variable declaration syntax in C.

```
%{
#include
%}
%%
(int|float|char|double)[_][a-zA-Z][a-zA-Z0-9_]*((([_]*,[_]*[a-zA-Z][a-zA-Z0-9_]*)*)[_]*; { printf("Valid Declaration: %s\n", yytext); }
.|
\n { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

37. Write a lex program to strip/remove all comments from a C source file.

```
%{
#include
%}
%%
"//".* { /* remove single line comment */ }
"/*"([^\]|\\*+[^*/])*\*+ "/" { /* remove multi line comment */ }
. { ECHO; }
\n { ECHO; }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

38. Write a lex program to identify valid Constants (Integer and String constants).

```
%{
#include
%}
%%
[0-9]+ { printf("Integer Constant: %s\n", yytext); }
\"[^\"]*\" { printf("String Constant: %s\n", yytext); }
.|
\n { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

39. Write a lex program to count nested brackets and balance check concepts.

```
%{
int op=0, cl=0;
%}
%%
"{" { op++; }
"}" { cl++; }
.|
\n { /* ignore */ }
%%
int main() {
    yylex();
    printf("Open braces: %d, Close braces: %d\n", op, cl);
    if(op == cl) printf("Balanced Braces\n");
    else printf("Imbalanced Braces\n");
    return 0;
}
int yywrap() { return 1; }
```

40. Write a lex program to count total number of Assignment Operators.

```
%{
int count=0;
}%
%%
"=" { count++; printf("Assignment operator found\n"); }
"=="|"+="|"-=" { /* ignore compound or relational */ }
.|
\n { /* ignore */ }
%%
int main() { yylex(); printf("Total Assignment Operators: %d\n", count); return 0; }
int yywrap() { return 1; }
```

41. Write a program to simulate a Shift-Reduce Parser.

```
# Python simulation of Shift-Reduce parsing for grammar: E -> E+E | id
stack = ""
input_str = "id+id$"
print(f"Stack\tInput\tAction")
while len(input_str) > 0:
    if "id" in stack:
        stack = stack.replace("id", "E")
        print(f"{stack}\t{input_str}\tReduce E->id")
    elif "E+E" in stack:
        stack = stack.replace("E+E", "E")
        print(f"{stack}\t{input_str}\tReduce E->E+E")
    else:
        if input_str[0] != '$':
            stack += input_str[0]
            input_str = input_str[1:]
            print(f"{stack}\t{input_str}\tShift")
        else:
            break
if stack == "E": print("Success / Parsing Complete")
```

42. Write a program to compute FIRST sets of given grammar rules.

```
# Concept logic for FIRST set
grammar = {'S': ['aAB', 'b'], 'A': ['c'], 'B': ['d']}
first = {}
for nt, prods in grammar.items():
    first[nt] = set()
    for p in prods:
        if p[0].islower():
            first[nt].add(p[0])
print("FIRST Sets:", first)
```

43. Write a program to compute FOLLOW sets of given grammar rules.

```
# Concept logic for FOLLOW set
grammar = {'S': ['aAB'], 'A': ['c'], 'B': ['d']}
follow = {'S': {'$'}}
# Follow of A is FIRST(B) = {d}
follow['A'] = {'d'}
# Follow of B is FOLLOW(S) = {$}
follow['B'] = {'$'}
print("FOLLOW Sets:", follow)
```

44. Write a program to check if an input sentence is Simple or Compound.

```
# Simple Rule-based NLP checker
sentence = "I went to the lab and I completed the code."
if " and " in sentence or " or " in sentence or " but " in sentence:
    print("Compound Sentence")
else:
    print("Simple Sentence")
```

45. Write a program to construct Quadruples intermediate code representation.

```
# Quadruples format: (operator, arg1, arg2, result)
# For expression: t1 = b * c
quadruple = [
    ('*', 'b', 'c', 't1'),
    ('+', 'a', 't1', 't2'),
    ('=', 't2', '-', 'x')
]
print("Quadruples Representation:")
for q in quadruple:
    print(f"{q[0]}\t{q[1]}\t{q[2]}\t{q[3]}")
```

46. Write a program to construct Triples intermediate code representation.

```
# Triples format: (operator, arg1, arg2) using indices as references
triples = [
    (0, '*', 'b', 'c'),
    (1, '+', 'a', '(0)'),
    (2, '=', 'x', '(1)')
]
print("Triples Representation:")
for t in triples:
    print(f"({t[0]})\t{t[1]}\t{t[2]}\t{t[3]}")
```

47. Write a program to perform Common Subexpression Elimination optimization.

```
# Code optimization example
# Before: a = b + c; d = b + c
expr1 = "b + c"
expr2 = "b + c"
if expr1 == expr2:
    print("Optimized expressions:")
    print("t1 = b + c")
    print("a = t1")
    print("d = t1")
```

48. Write a program to simulate Loop Invariant Code Motion.

```
# Before optimization:
# for i in range(10): x = a + b; c[i] = x * i
print("Before Optimization: inside loop 'x = a + b'")
print("\nOptimized Code (Code Motion):")
print("x = a + b # Moved outside loop")
print("for i in range(10): c[i] = x * i")
```

49. Write a program to implement Constant Folding optimization.

```
# Optimization technique
a = "3.14 * 2 * r"
# Constant folding converts 3.14 * 2 into 6.28 dynamically
folded_constant = 3.14 * 2
print(f"Original expression: 3.14 * 2 * r")
print(f"Optimized expression: {folded_constant} * r")
```

50. Write a program to implement Dead Code Elimination.

```
# Dead code removal optimization
code_lines = [
    "a = 10",
    "b = 20",
    "return b",
    "c = a + b # Dead Code (unreachable)"
]
print("Optimized Active Code:")
for line in code_lines:
    print(line)
    if "return" in line:
        break
```

51. Write a conceptual Yacc parser framework for a Calculator.

```
%{
#include
void yyerror(char *s);
}%
%token NUM
%%
exp: exp '+' exp { $$= $1 + $3; }
    | exp '-' exp { $$ = $1 - $3; }
    | NUM        { $$ = $1; }
;
%%
```

52. Write a Yacc framework to validate C Variable Declarations.

```
%token TYPE ID
%%
start: TYPE list ';'
list: ID ',' list | ID;
%%
void yyerror(char *s) { printf("Invalid declaration!\n"); }
```

53. Write a Yacc framework to validate language string $a^n b^n$.

```
%%
S: 'a' S 'b' | /* epsilon */ ;
%%
// Validates equal number of 'a's and 'b's matching grammar rules
```

54. Write a Yacc structure to recognize mathematical bracket matching.

```
%%
expr: '(' expr ')' | NUM;
%%
// Checks syntax parsing correctness of nested arithmetic blocks
```

55. Write a program to simulate Subset Construction (NFA to DFA concept).

```
# Conceptual logic for Subset construction
nfa_states = {'q0', 'q1'}
# Epsilon closure of starting state
eps_closure = {'q0'}
print("DFA Start State calculated from E-closure:", eps_closure)
```

56. Write a program to simulate State Minimization of a DFA.

```
# State partitioning approach
states = {'A', 'B', 'C', 'D', 'E'}
accepting = {'E'}
non_accepting = states - accepting
print("Initial Partition P0:", [non_accepting, accepting])
```

57. Write a program to check for Syntactic Errors in custom inputs.

```
# Basic Parser error check simulation
input_tokens = ['int', 'a', '='] # Missing final identifier/semicolon
if input_tokens[-1] == '=':
    print("Syntax Error: Expected expression after '='")
```

58. Write a code concept simulating a Turing Machine accepting $0^n 1^n$.

```
# Python Turing Machine Transition simulation snippet
head_pos = 0
tape = ['0', '0', '1', '1', 'B']
state = 'q0'
# logic loops through changing 0->X and 1->Y until reaching Blank state 'B'
print("Tape matches structure  $0^n 1^n$ ")
```

59. Write a program to generate Target Assembly Code from 3AC.

```
# Converting intermediate code: t0 = a + b into Assembly instructions
tac = ("+", "a", "b", "t0")
print("Generated Assembly Language Instructions:")
print(f"MOV R0, {tac[1]}")
print(f"ADD R0, {tac[2]}")
print(f"MOV {tac[3]}, R0")
```

60. Write a template structure showing joint integration of Lex & Yacc.

```
/* Lex File part */
// %%
// [0-9]+ { yyval = atoi(yytext); return NUM; }

/* Yacc File part */
// %%
// program: expr { printf("Result: %d\n", $1); };
```